

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
advaith@DESKTOP-0E1OBMR:~$ mkdir PROCESSID
advaith@DESKTOP-0E1OBMR:~$ nano PROCESSID.c
advaith@DESKTOP-0E1OBMR:~$ nano PROCESSID.c
advaith@DESKTOP-0E1OBMR:~$ gcc PROCESSID.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
My Process ID (PID) = 654
```

OUTPUTS: advaith@DESKTOP-0E1OBMR:~\$ |

```
advaith@DESKTOP-0E1OBMR:~$ nano CHILDPROCESS.c
advaith@DESKTOP-0E1OBMR:~$ gcc CHILDPROCESS.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Parent Process
PID = 919
Child Process
PID = 920
```

```
PID = 920
advaith@DESKTOP-0E1OBMR:~$ mkdir PARENTPID
advaith@DESKTOP-0E1OBMR:~$ nano PARENTPID.c
advaith@DESKTOP-0E1OBMR:~$ gcc PARENTPID.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Child Process Executing...
Child Finished
Parent Process Continues
advaith@DESKTOP-0E1OBMR:~$ |
```

```
Parent Process Continues
advaith@DESKTOP-0E1OBMR:~$ gcc MULTIPLECHILD.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Child 1 PID = 1081
Child 2 PID = 1082
Child 3 PID = 1083
advaith@DESKTOP-0E1OBMR:~$ |
```